The two workloads, and the lifecycle the mixed one walks

A fixed draw order makes the per-operation-kind counts identical in both languages, so the workload doubles as a parity signal

signal: the steady-state hot path

Preloads a population of connected UEs and then hammers handle_signaling, isolating per-message latency with no lifecycle transitions in the way.

mixed: the full RAN lifecycle

A pinned generator that walks each UE through a realistic procedure sequence, using a per-slot expected-state model so that every operation it emits is valid for that UE.

The procedure sequence the mixed workload walks, per UE

create	setup	signal	add_drb	begin	complete / abort	release
Idle	Connected	Connected	Connected	Reconfiguring	Connected	Released

resulting state

Control parameters

5 000 active UEs, and 14 offered-load levels up to 784 kops/s swept from nominal through saturation into overload. Open loop against a fixed intended send schedule, so latency is measured from the time an operation should have been sent rather than was.

Why the draw order is pinned

A fixed SplitMix64 draw order means that for a given seed and active-UE population the two implementations emit exactly the same count of each operation kind. A divergence is therefore a parity failure rather than a difference in what the two were asked to do.

The workload is synthetic and specification-justified rather than replayed from production traffic, so its operation mix is plausible rather than validated.